

equity_pricing

September 20, 2026

1 PROTOTYPE EQUITY PRICING

One of the basic functions of OSEM is creating and manipulating cash flows from different classes of financial instruments. This page shows the processing of company shares held within a portfolio.

An ownership interest or a share represents ownership of a small piece of a legal entity. The most common of them being listed companies. In OSEM these shares provide two kinds of benefits. A periodic dividend and its intrinsic value (The share can be sold to another entity).

The enterprise and with it the equity share is assumed to grow at a constant rate every year. At periodic periods, the dividend is paid out. The size of the dividend determined by the value of the share. A % of the market value referred here as dividend yield.

This page has 4 sections.

- 1) The first section shows how to import the necessary information about equity positions, the overall economic environment as well as other parameters.
- 2) The next section imports the term structure which is needed later on.
- 3) The third section shows how to generate the cash flows for a set of equities.
- 4) The last section shows how a hypothetical equity can be calibrated if the user wishes to perform a risk-neutral run.

What a projection run actually executes. Sections 1 to 3 follow the path a run takes: `main.py` builds the same dividend and terminal cash flows and lets them expire period by period. The pricing at the end of section 3 and the calibration in section 4 do not. In a projection an equity price moves with the growth rate alone,

```
eq_price_df[current_date] = eq_price_df[previous_date] * (1 + growth) **  
time_frac (src/osem/main.py:316),
```

so no equity is ever priced by discounting its cash flows, and the three spreads introduced below are not read anywhere in `main.py` or `MainLoop.py`. `price_share` is reachable only from `bisection_growth`, and nothing in `src/` or `unit_tests/` calls that. What follows is a demonstration of machinery that exists and works, not of a step the model currently performs.

1.1 Importing and handling of equity data

The packages and imports needed in this script are the following:

```
[1]: import datetime  
import os  
import pandas as pd
```

```

from pathlib import Path
import sys

base_folder = next(path for path in [Path.cwd(), *Path.cwd().parents] if (path /
↳ "ALM.ini").is_file())
sys.path.insert(0, str(base_folder / "src"))
os.chdir(base_folder)

from osem.ConfigurationClass import Configuration
from osem.CurvesClass import Curves
from osem.EquityClasses import *
from osem.ImportData import get_equity_share, get_settings, import_SWEiopa, \
    get_configuration
from osem.MainLoop import create_cashflow_dataframe

```

Set up the base folder

```
[2]: base_folder = os.getcwd() # Get current working directory
```

Most of the run settings are saved in the configuration file:

```
[3]: conf: Configuration
conf = get_configuration(os.path.join(base_folder, "ALM.ini"), os)
```

These lines of code just extract the absolute location of different files:

```
[4]: parameters_file = conf.input_parameters
cash_portfolio_file = conf.input_cash_portfolio
equity_portfolio_file = conf.input_equity_portfolio
```

The settings object holds data about file locations, information about the run settings and model parameters such as modelling date.

```
[5]: settings = get_settings(parameters_file)
```

The EquityShare object contains information about each equity position. This includes: * Asset_ID * NACE * Issuer * Issue_Date * Dividend_Yield * Frequency * Units * Market_Price * Growth_Rate * Spread_Country * Spread_Sector * Spread_Stress

Note that Dividend_Yield is the yield paid on **each** dividend date, not an annualised figure: Frequency sets only the spacing of the dividend dates. A share quoted with an annual yield paid f times a year is entered with Dividend_Yield divided by f . The three spreads are used by price_share in the pricing section below.

```
[6]: equity_input_generator = get_equity_share(equity_portfolio_file)
equity_input = {equity_share.asset_id: equity_share for equity_share in
↳ equity_input_generator}
```

EquitySharePortfolio object contains all EquityShare objects in a dictionary.

```
[7]: equity_portfolio = EquitySharePortfolio(equity_input)
```

1.2 Importing the information about the economic environment

`import_SWEiopa()` reads the necessary data about the current yield curve. Inside OSEM, the parameters related to the yield curve are saved in the `Curves` object.

The ultimate forward rate (`ufr`) is read here and handed to `create_terminal_flows` as `terminal_rate`, but it does not currently affect the result. `terminal_amount` in `EquityClasses.py` returns the grown market price, with the Gordon growth formula `market_price / (terminal_rate - growth_rate)` left commented out, so the terminal value is simply the market value of the share at the end of the modelling window. `main.py` passes the `ufr` in the same way and it is ignored there too.

```
[8]: [maturities_country, curve_country, extra_param, Qb] = import_SWEiopa(settings.  
      ↪EIOPA_param_file,  
  
      ↪settings.EIOPA_curves_file, settings.country)  
# Curves object with information about term structure  
curves = Curves(extra_param["UFR"] / 100, settings.precision, settings.tau, ↪  
      ↪settings.modelling_date,  
               settings.country)
```

```
[9]: ufr = extra_param["UFR"]/100 # ultimate forward rate  
precision = float(settings.precision) # Numeric precision of the optimisation  
# Targeted distance between the extrapolated curve and the ufr at the ↪  
      ↪convergence point  
tau = float(settings.tau) # 1 basis point
```

```
[10]: curves.set_observed_term_structure(maturity_vec=curve_country.index.  
      ↪to_numpy(dtype=float), yield_vec=curve_country.values)  
curves.calc_fwd_rates()
```

```
[11]: # The +1 matches main.py: the projected curve has to reach one year past the ↪  
      ↪end of the  
# modelling window so the last projection period can still be priced.  
curves.project_forward_rate(settings.n_proj_years+1)
```

```
[12]: curves.calibrate_projected(settings.n_proj_years+1, 0.05, 0.5, 1000)
```

1.3 Projection of cash flows for an equity portfolio

The first computation step inside the OSEM equity preparation process is the identification of all the unique dates and dividend size amounts. The representation of assets in terms of individual cash flows on the time-line is one of the core principles of OSEM. This is done by two functions. One for dividend dates and another for terminal rates.

Both functions generate a list of dictionaries containing the date of a cash flow and the amount. Same is also true for the terminal amount calculation.

Calculation of the dividend amount: The dividend size is calculated using the dividend yield provided as input for each equity position. However the market value changes as time moves forward. To account for this, the growth rate and the time fraction between the modelling date and the date of the cash flow is used to calculate a future market value.

Write MP for the market price at the modelling date MD , g for the annual growth rate, dy for the dividend yield, and

$$t_i = \frac{t_i^d - MD}{365.25}$$

for the year fraction from the modelling date to the i -th dividend date. The market value grown to that date, and the dividend paid out of it, are

$$MV(t_i) = MP \cdot (1 + g)^{t_i} \quad \quad \quad div_i = MV(t_i) \cdot dy$$

The same logic gives the terminal value, evaluated at the end of the modelling window E :

$$term = MV(T) = MP \cdot (1 + g)^T, \quad T = \frac{E - MD}{365.25}$$

The terminal value is the grown market value itself: no yield is applied to it, and it is not discounted. The share is assumed to be sold at its market value at the end of the modelling window.

```
[13]: dividend_flows = equity_portfolio.create_dividend_flows(settings.
      ↪modelling_date, settings.end_date)
      terminal_flows = equity_portfolio.create_terminal_flows(modelling_date=settings.
      ↪modelling_date,
      ↪terminal_date=settings.end_date,
      ↪terminal_rate=curves.ufr)
```

All cash flows can be represented in a matrix with all possible cash flow dates as columns and all equities as rows. The non-zero entries then represent the value of the cash flow at that date. The first step is to calculate the unique dates for the entire portfolio of equities. This is done by the `unique_dates_profiles()` function.

The same logic can be applied to terminal dates.

Both can then conveniently be represented as DataFrames.

Note that a vector of growth rates is also provided as output. This makes it simpler to increase the market value of the portfolio as OSEM moves from one modelling period to the next one.

```
[14]: unique_list = equity_portfolio.unique_dates_profile(dividend_flows)
      unique_terminal_list = equity_portfolio.unique_dates_profile(terminal_flows)
```

```
[15]: [market_price_df, growth_rate_df, units_df] = equity_portfolio.
      ↪init_equity_portfolio_to_dataframe(settings.modelling_date)
```

The `create_cashflow_dataframe()` function converts the list of dictionaries of cashflows and dates, into a single DataFrame:

```
[16]: cash_flows = create_cashflow_dataframe(dividend_flows, unique_list)
      # Dataframe with terminal cash flows
      terminal_cash_flows = create_cashflow_dataframe(terminal_flows,
      ↪unique_terminal_list)
```

```
[17]: display(cash_flows)
```

	2023-12-03	2024-12-03	2025-12-03	2026-12-03	2027-12-03	2028-12-03	\
1125	3.712281	3.749481	3.786950	3.824793	3.863015	3.901725	
2123	0.613205	0.625494	0.637996	0.650747	0.663753	0.677055	
3232	2.315166	2.407967	2.504218	2.604317	2.708417	2.816981	
1	2.323760	2.347045	2.370499	2.394188	2.418114	2.442345	
2	3.946368	4.025460	4.105913	4.187975	4.271676	4.357287	
3	0.050365	0.052384	0.054478	0.056656	0.058920	0.061282	
4	0.094158	0.095101	0.096051	0.097011	0.097981	0.098963	
5	2.282822	2.328574	2.375113	2.422582	2.471000	2.520523	
6	3.930951	4.088518	4.251945	4.421904	4.598656	4.782988	
7	2.836797	2.865224	2.893856	2.922775	2.951983	2.981564	
8	4.654691	4.747978	4.842872	4.939662	5.038387	5.139364	
9	3.930951	4.088518	4.251945	4.421904	4.598656	4.782988	
10	2.836797	2.865224	2.893856	2.922775	2.951983	2.981564	
11	4.654691	4.747978	4.842872	4.939662	5.038387	5.139364	
12	3.930951	4.088518	4.251945	4.421904	4.598656	4.782988	
13	2.836797	2.865224	2.893856	2.922775	2.951983	2.981564	
14	4.654691	4.747978	4.842872	4.939662	5.038387	5.139364	
15	3.930951	4.088518	4.251945	4.421904	4.598656	4.782988	
16	2.836797	2.865224	2.893856	2.922775	2.951983	2.981564	
17	4.654691	4.747978	4.842872	4.939662	5.038387	5.139364	
18	3.930951	4.088518	4.251945	4.421904	4.598656	4.782988	
19	2.836797	2.865224	2.893856	2.922775	2.951983	2.981564	
20	4.654691	4.747978	4.842872	4.939662	5.038387	5.139364	
21	3.930951	4.088518	4.251945	4.421904	4.598656	4.782988	
22	2.836797	2.865224	2.893856	2.922775	2.951983	2.981564	
23	4.654691	4.747978	4.842872	4.939662	5.038387	5.139364	
24	3.930951	4.088518	4.251945	4.421904	4.598656	4.782988	
25	2.836797	2.865224	2.893856	2.922775	2.951983	2.981564	
26	4.654691	4.747978	4.842872	4.939662	5.038387	5.139364	
27	3.930951	4.088518	4.251945	4.421904	4.598656	4.782988	
28	2.836797	2.865224	2.893856	2.922775	2.951983	2.981564	
29	4.654691	4.747978	4.842872	4.939662	5.038387	5.139364	
30	3.957567	4.116201	4.280734	4.451844	4.629793	4.815373	
	2029-12-03	2030-12-03	2031-12-03	2032-12-03	...	2063-12-03	\
1125	3.940715	3.980095	4.019869	4.060150	...	5.527081	
2123	0.690587	0.704389	0.718467	0.732866	...	1.353981	

3232	2.929581	3.046683	3.168465	3.295469	...	11.115161
1	2.466751	2.491402	2.516299	2.541514	...	3.459761
2	4.444372	4.533198	4.623800	4.716467	...	8.713738
3	0.063732	0.066279	0.068928	0.071691	...	0.241805
4	0.099951	0.100950	0.101959	0.102981	...	0.140188
5	2.570898	2.622281	2.674690	2.728295	...	5.040562
6	4.974174	5.173002	5.379777	5.595419	...	18.872575
7	3.011359	3.041452	3.071845	3.102627	...	4.223605
8	5.242080	5.346849	5.453712	5.563013	...	10.277742
9	4.974174	5.173002	5.379777	5.595419	...	18.872575
10	3.011359	3.041452	3.071845	3.102627	...	4.223605
11	5.242080	5.346849	5.453712	5.563013	...	10.277742
12	4.974174	5.173002	5.379777	5.595419	...	18.872575
13	3.011359	3.041452	3.071845	3.102627	...	4.223605
14	5.242080	5.346849	5.453712	5.563013	...	10.277742
15	4.974174	5.173002	5.379777	5.595419	...	18.872575
16	3.011359	3.041452	3.071845	3.102627	...	4.223605
17	5.242080	5.346849	5.453712	5.563013	...	10.277742
18	4.974174	5.173002	5.379777	5.595419	...	18.872575
19	3.011359	3.041452	3.071845	3.102627	...	4.223605
20	5.242080	5.346849	5.453712	5.563013	...	10.277742
21	4.974174	5.173002	5.379777	5.595419	...	18.872575
22	3.011359	3.041452	3.071845	3.102627	...	4.223605
23	5.242080	5.346849	5.453712	5.563013	...	10.277742
24	4.974174	5.173002	5.379777	5.595419	...	18.872575
25	3.011359	3.041452	3.071845	3.102627	...	4.223605
26	5.242080	5.346849	5.453712	5.563013	...	10.277742
27	4.974174	5.173002	5.379777	5.595419	...	18.872575
28	3.011359	3.041452	3.071845	3.102627	...	4.223605
29	5.242080	5.346849	5.453712	5.563013	...	10.277742
30	5.007853	5.208027	5.416203	5.633305	...	19.000358

	2064-12-03	2065-12-03	2066-12-03	2067-12-03	2068-12-03	2069-12-03	\
1125	5.582466	5.638252	5.694596	5.751503	5.809137	5.867188	
2123	1.381117	1.408720	1.436875	1.465592	1.494965	1.524844	
3232	11.560698	12.022803	12.503380	13.003166	13.524382	14.064979	
1	3.494430	3.529351	3.564620	3.600242	3.636318	3.672656	
2	8.888374	9.066019	9.247214	9.432030	9.621062	9.813350	
3	0.251497	0.261550	0.272005	0.282878	0.294216	0.305977	
4	0.141593	0.143007	0.144437	0.145880	0.147342	0.148814	
5	5.141583	5.244343	5.349158	5.456067	5.565414	5.676646	
6	19.629059	20.413674	21.229651	22.078244	22.963223	23.881111	
7	4.265928	4.308558	4.351614	4.395100	4.439142	4.483503	
8	10.483724	10.693253	10.906970	11.124959	11.347919	11.574721	
9	19.629059	20.413674	21.229651	22.078244	22.963223	23.881111	
10	4.265928	4.308558	4.351614	4.395100	4.439142	4.483503	
11	10.483724	10.693253	10.906970	11.124959	11.347919	11.574721	
12	19.629059	20.413674	21.229651	22.078244	22.963223	23.881111	

13	4.265928	4.308558	4.351614	4.395100	4.439142	4.483503
14	10.483724	10.693253	10.906970	11.124959	11.347919	11.574721
15	19.629059	20.413674	21.229651	22.078244	22.963223	23.881111
16	4.265928	4.308558	4.351614	4.395100	4.439142	4.483503
17	10.483724	10.693253	10.906970	11.124959	11.347919	11.574721
18	19.629059	20.413674	21.229651	22.078244	22.963223	23.881111
19	4.265928	4.308558	4.351614	4.395100	4.439142	4.483503
20	10.483724	10.693253	10.906970	11.124959	11.347919	11.574721
21	19.629059	20.413674	21.229651	22.078244	22.963223	23.881111
22	4.265928	4.308558	4.351614	4.395100	4.439142	4.483503
23	10.483724	10.693253	10.906970	11.124959	11.347919	11.574721
24	19.629059	20.413674	21.229651	22.078244	22.963223	23.881111
25	4.265928	4.308558	4.351614	4.395100	4.439142	4.483503
26	10.483724	10.693253	10.906970	11.124959	11.347919	11.574721
27	19.629059	20.413674	21.229651	22.078244	22.963223	23.881111
28	4.265928	4.308558	4.351614	4.395100	4.439142	4.483503
29	10.483724	10.693253	10.906970	11.124959	11.347919	11.574721
30	19.761964	20.551891	21.373393	22.227732	23.118703	24.042806

	2070-12-03	2071-12-03	2072-12-03
1125	5.925819	5.985037	6.045011
2123	1.555319	1.586404	1.618198
3232	14.627186	15.211865	15.821613
1	3.709358	3.746426	3.783967
2	10.009482	10.209533	10.414147
3	0.318207	0.330927	0.344191
4	0.150301	0.151803	0.153324
5	5.790100	5.905822	6.024184
6	24.835688	25.828423	26.863723
7	4.528307	4.573559	4.619389
8	11.806055	12.042013	12.283353
9	24.835688	25.828423	26.863723
10	4.528307	4.573559	4.619389
11	11.806055	12.042013	12.283353
12	24.835688	25.828423	26.863723
13	4.528307	4.573559	4.619389
14	11.806055	12.042013	12.283353
15	24.835688	25.828423	26.863723
16	4.528307	4.573559	4.619389
17	11.806055	12.042013	12.283353
18	24.835688	25.828423	26.863723
19	4.528307	4.573559	4.619389
20	11.806055	12.042013	12.283353
21	24.835688	25.828423	26.863723
22	4.528307	4.573559	4.619389
23	11.806055	12.042013	12.283353
24	24.835688	25.828423	26.863723
25	4.528307	4.573559	4.619389

26	11.806055	12.042013	12.283353
27	24.835688	25.828423	26.863723
28	4.528307	4.573559	4.619389
29	11.806055	12.042013	12.283353
30	25.003847	26.003303	27.045613

[33 rows x 50 columns]

```
[18]: display(terminal_cash_flows)
```

```

2073-04-29
1125 202.308916
2123 32.622931
3232 401.833450
1 126.638375
2 209.949558
3 8.741690
4 5.131321
5 121.447744
6 682.278232
7 154.597497
8 247.632811
9 682.278232
10 154.597497
11 247.632811
12 682.278232
13 154.597497
14 247.632811
15 682.278232
16 154.597497
17 247.632811
18 682.278232
19 154.597497
20 247.632811
21 682.278232
22 154.597497
23 247.632811
24 682.278232
25 154.597497
26 247.632811
27 682.278232
28 154.597497
29 247.632811
30 686.897824

```

Risk spreads In OSEM, the cash flows of equity shares increase with the assumed growth rate. However in real world applications, when calculating the present value of these cash flows, the “riskiness” of the issuer should be accounted for. This is done using discounting spreads. These

spreads represent the fact that certain geographies and certain sectors are more prone to failure and the required risk premium is therefore higher.

Each equity position has 3 kinds of spreads are available:

- 1) Country specific spread: Country specific spread represents the extra riskiness related to the country of operation (`spread_country`)
- 2) Sector specific spread: Spread specific to the sector in which the issuer operates (`spread_sector`)
- 3) Assumed shock scenario specific spread: Spread specific to the shock scenario (`spread_stress`)

These three fields are inputs to `price_share` only. A projection run never reads them, so changing them in `Equity_Portfolio.csv` has no effect on a run's results.

Calculation of present value of each instrument The cashflows can be used to price the current market value of the share, implied by the assumed economic parameters.

Note that this pricing is done using the risk free rate as the discounting factor with a risk spread equal to the sum of the 3 components. This combined spread is added to the yield in a parallel shift. To demonstrate, the discount factors are calculated as:

$$df_i(t) = \frac{1}{(1 + y(t) + spreadCountry + spreadSector + spreadShock)^t}$$

Where: - t is the time period - $df_i(t)$ is the discount factor at time t for the equity i - $y(t)$ is the yield implied by the risk-free interest rate at time t - $spreadCountry$, $spreadSector$, $spreadShock$ are the country, sector and shock spreads.

This example will show pricing at the modelling date.

```
[19]: proj_period = 0
```

```
[20]: asset_id_tmp = cash_flows.index[0]
```

```
[21]: display(equity_portfolio.equity_share[asset_id_tmp])
```

```
EquityShare(asset_id=1125, nace='A1.4.5', issuer=None, issue_date=datetime.
↳date(2021, 12, 3), dividend_yield=0.03, frequency=1, units=100.0,↳
↳market_price=123.01, growth_rate=0.01, spread_country=0.0, spread_sector=0.0,↳
↳spread_stress=0.0)
```

```
[22]: asset_id_tmp = cash_flows.index[0]

temp_dividend = cash_flows.loc[asset_id_tmp].to_dict()

temp_terminal = terminal_cash_flows.loc[asset_id_tmp].to_dict()

price = equity_portfolio.equity_share[asset_id_tmp].price_share(temp_dividend,↳
↳temp_terminal, settings.modelling_date, proj_period, curves)
print(price)
```

169.9570978329776

C:\Users\grego\anaconda3\Lib\site-packages\numpy\core\fromnumeric.py:86:

FutureWarning: The behavior of DataFrame.sum with axis=None is deprecated, in a future version this will reduce over both axes and return a scalar. To retain the old behavior, pass axis=0 (or do not pass axis)

return reduction(axis=axis, out=out, **passkwargs)

C:\Users\grego\OneDrive\Documenti\GitHub\Open_Source_Economic_Model\src\osem\EquityClasses.py:256: FutureWarning: Calling float on a single element Series is deprecated and will raise a TypeError in the future. Use float(ser.iloc[0]) instead

disc_value: float = float(np.sum(nodisc_value))

Additivity of spreads The 3 kinds of spread work in the same way and increasing any of them has the same effect.

We will demonstrate this by fixing them one at a time.

```
[23]: asset_id_tmp = cash_flows.index[0]

temp_dividend = cash_flows.loc[asset_id_tmp].to_dict()

temp_terminal = terminal_cash_flows.loc[asset_id_tmp].to_dict()
```

```
[24]: equity_portfolio.equity_share[asset_id_tmp].spread_country = 0.01
equity_portfolio.equity_share[asset_id_tmp].spread_sector = 0.01
equity_portfolio.equity_share[asset_id_tmp].spread_stress = 0.00
```

```
[25]: price = equity_portfolio.equity_share[asset_id_tmp].price_share(temp_dividend,
↳ temp_terminal, settings.modelling_date, proj_period, curves)
print(price)
```

102.12556437974273

```
[26]: equity_portfolio.equity_share[asset_id_tmp].spread_country = 0.01
equity_portfolio.equity_share[asset_id_tmp].spread_sector = 0
equity_portfolio.equity_share[asset_id_tmp].spread_stress = 0.01
```

```
[27]: price2 = equity_portfolio.equity_share[asset_id_tmp].price_share(temp_dividend,
↳ temp_terminal, settings.modelling_date, proj_period, curves)
print(price2)
```

102.12556437974273

```
[28]: equity_portfolio.equity_share[asset_id_tmp].spread_country = 0
equity_portfolio.equity_share[asset_id_tmp].spread_sector = 0
equity_portfolio.equity_share[asset_id_tmp].spread_stress = 0.02
```

```
[29]: price3 = equity_portfolio.equity_share[asset_id_tmp].price_share(temp_dividend,
    ↪temp_terminal, settings.modelling_date, proj_period, curves)
    print(price3)
```

102.12556437974273

1.4 Calibration of an equity share

In a real world run, the growth rate of each equity is brought in as a modelling assumption and must be calculated externally. If the user is interested in a risk-neutral run, the performance of each asset must equal to that of the risk free rate. In OSEM this is done using the growth rate as the calibrating parameter. To do this, a bisection algorithm is used to calibrate the growth rate such that the discounted cash flows of each equity equal to the current market price.

For this demonstration, a single equity position is created, the growth rate is calibrated and then the present value of the calibrated equity is compared to the assumed current market value.

Note that the projection period is selected as 0 meaning the initial modelling date. But the calibration works also for other projection periods. The only change needed would be to change the market price.

Note that this calibration is not wired into a projection. `run_type` is read from `Parameters.csv` into the settings object and never consulted again, and growth rates are taken from the input file as given, so nothing currently switches a run to risk-neutral. The bisection below is the machinery such a run would need, demonstrated on a single share.

```
[30]: proj_period = 0
```

```
[31]: spread_country = 0
    spread_sector = 0.02
    spread_stress = 0
```

```
[32]: test_share_1 = EquityShare(asset_id=1, nace='A1.4.5', issuer=None,
    ↪issue_date=datetime.date(2021, 12, 3), dividend_yield=0.03, frequency=1,
    ↪units=1, market_price=94.0, growth_rate=0.
    ↪01, spread_country=spread_country, spread_sector=spread_sector, spread_stress=spread_stress,
    ↪)
```

```
[33]: opt_growth = test_share_1.bisection_growth(-1, 1, settings.modelling_date,
    ↪settings.end_date, proj_period, curves, 0.00000001, 100000)
```

```
[34]: print(opt_growth)
```

0.017648451030254364

```
[35]: test_dividends = test_share_1.create_single_cash_flows(settings.modelling_date,
    ↪settings.end_date, opt_growth)
    test_terminal = test_share_1.create_single_terminal(settings.modelling_date,
    ↪settings.end_date, curves.ufr, opt_growth)
```

If the calibration was performed correctly, the present value calculated by discounting future cash flows with the assumed risk free rate should be equal to the initial observed market price.

```
[36]: print(test_share_1.market_price)
```

94.0

```
[37]: print(test_share_1.price_share(test_dividends, test_terminal, settings.  
    ↪modelling_date, proj_period, curves))
```

94.00001724597202